

Архитектура вычислительных систем и компьютерных сетей

Вошинская Гильда Эдгаровна

Характеристики систем памяти

- местоположение;
- ёмкость;
- единица пересылки;
- метод доступа;
- быстродействие;
- физический тип;
- физические особенности;
- стоимость.

По месту расположения ЗУ

разделяют на

- процессорные,
- внутренние,
- внешние.

Ёмкость ЗУ

характеризуют числом битов либо байтов, которое может храниться в ЗУ.

На практике применяются более крупные единицы:

- кило (10^3),
- мега (10^6),
- гига (10^9),
- тера (10^{12}),
- пета (10^{15}),
- экса (10^{18}).

Ёмкость ЗУ

В вычислительной технике, ориентированной на двоичную систему счисления, они соответствуют значениям достаточно близким, но представляющим собой степени 2, т.е.

2^{10} , 2^{20} , 2^{30} , 2^{40} , 2^{50} , 2^{60} .

Во избежание разночтений, в последнее время международные организации по стандартизации, например IEEE, предлагают ввести новые обозначения, добавив слово binary:

«кибибайт», «мебибайт» и т.д.

Для обозначения новых единиц предлагаются обозначения Ki, Mi, Gi, Ti, Pi, Ei.

Единица пересылки

Для основной памяти единица пересылки определяется *шириной шины данных*,

т.е. количеством битов, передаваемых по линиям шины параллельно.

Обычно единица пересылки равна длине слова.

Применительно к внешней памяти данные часто передаются единицами, превышающими размер слова, и такие единицы называются блоками.

Методы доступа

- Последовательный доступ;
- прямой доступ;
- произвольный доступ;
- ассоциативный доступ.

Быстродействие ЗУ

один из важнейших показателей.

Для количественной оценки быстродействия используют три параметра:

- время доступа (T_d);*
- длительность цикла памяти или период обращения ($T_{ц}$);*
- скорость передачи.*

Время доступа

для памяти с произвольным доступом оно соответствует интервалу времени от момента поступления адреса до момента, когда данные заносятся в память или становятся доступными.

В ЗУ с подвижным носителем информации это время установки головок записи/считывания.

Длительность цикла памяти

Понятие применяется к памяти с произвольным доступом и означает минимальное время между двумя последовательными обращениями к памяти.

Период обращения включает в себя время доступа плюс некоторое дополнительное время.

Скорость передачи

это скорость, с которой данные могут передаваться в память или из памяти.

Для памяти с произвольным доступом она равна $1/T_{\text{ц}}$.

Для других видов памяти скорость передачи определяется

$$T_N = T_A + N/R, \text{ где}$$

T_N – среднее время считывания или записи N байтов;

T_A – среднее время доступа;

R – скорость пересылки в битах в секунду.

Физический тип ЗУ

Три наиболее распространенных технологии ЗУ:

- полупроводниковая память;
- память с магнитным носителем информации;
- память с оптическим носителем.

В зависимости от применённой технологии нужно учитывать некоторые физические особенности ЗУ, например *энергозависимость*.

Оперативная память

Основной единицей хранения данных в памяти является двоичный разряд, который называется *битом*.

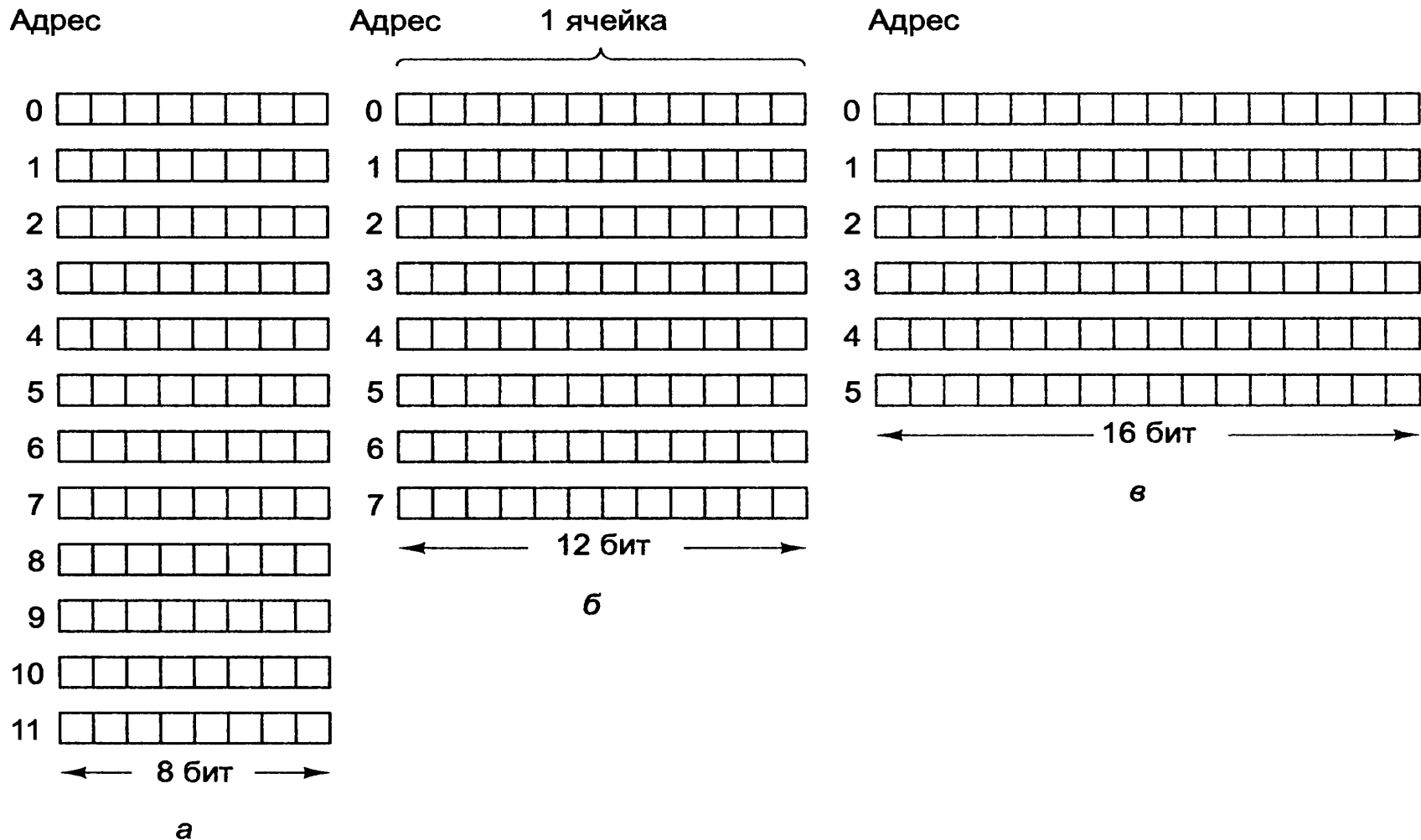
Бит может содержать 0 или 1. Эта самая маленькая единица памяти.

В двоичной системе требуется различать всего две величины, следовательно, это — самый надежный метод кодирования цифровой информации.

Адреса памяти

Память состоит из ячеек, каждая из которых может хранить некоторую порцию информации. Каждая ячейка имеет номер, который называется *адресом*. По адресу программы могут ссылаться на определенную ячейку. Если память содержит n ячеек, они будут иметь адреса от 0 до $n - 1$. Все ячейки памяти содержат одинаковое число бит. Если ячейка состоит из k бит, она может содержать любую из 2^k комбинаций.

Примеры организации 96-разрядной памяти



Число бит в ячейке памяти некоторых моделей компьютеров

Компьютер	Число битов в ячейке
Burroughs B1700	1
IBM PC	8
DEC PDP-8	12
IBM 1130	16
DEC PDP-15	18
XDS 940	24
Electrologica X8	27
XDS Sigma 9	32
Honeywell 6180	36
CDC 3600	48
CDC Cyber	60

Ячейка — минимальная адресуемая единица памяти

В последние годы практически все производители выпускают компьютеры с 8-разрядными ячейками, которые называются *байтами*. Байты группируются в слова.

Такая единица как слово, необходима, поскольку большинство команд производят операции над целыми словами.

Кэш-память

Процессоры всегда работали быстрее, чем память.

На практике такое несоответствие в скорости работы приводит к тому, что, когда процессор обращается к памяти, проходит несколько машинных циклов, прежде чем он получит запрошенное слово.

Инженеры знают, как создать память, которая работает так же быстро, как процессор. Ее приходится помещать прямо на микросхему процессора (поскольку информация через шину поступает очень медленно).

Размещение памяти большого объема на микросхеме процессора делает его больше и, следовательно, дороже.

Память небольшого объема с высокой скоростью работы называется *кэш-памятью*.

Основная память и кэш-память делятся на
блоки фиксированного размера
с учетом принципа локальности.
Блоки внутри кэш-памяти обычно
называют
строками кэша (cache lines).
Кэш-память очень важна для
высокопроизводительных процессоров.

При создании кэш-памяти нужно решить несколько проблем

1. Объем кэш-памяти. Чем больше объем, тем лучше работает память, но тем дороже она стоит.
2. Размер строки кэша.
3. Механизм организации кэш-памяти, т.е. то, как она определяет, какие именно слова находятся в ней в данный момент.
4. Должны ли команды и данные находиться вместе в общей кэш-памяти.
5. Количество блоков кэш-памяти.

При создании кэш-памяти нужно решить несколько проблем

Проще всего разработать объединенную кэш-память (unified cache), в которой будут храниться и данные и команды. Тогда вызов команд и данных автоматически уравнивается.

Однако, в настоящее время существует тенденция к использованию разделенной кэш-памяти (split cache), когда команды хранятся в одной кэш-памяти, а данные — в другой. Такая архитектура также называется гарвардской (Harvard architecture).

Причины разделения кэш-памяти

1. При конвейерной организации должна быть возможность одновременного доступа и к командам, и к данным (операндам). Разделенная кэш-память позволяет осуществлять параллельный доступ, а общая — нет.
2. Поскольку команды обычно не меняются во время выполнения программы, содержание кэша команд не приходится записывать обратно в основную память.

Уровни кэш-памяти

В настоящее время очень часто кэш-память первого уровня располагается прямо на микросхеме процессора, кэш-память второго уровня — не на самой микросхеме, но в корпусе процессора, а кэш-память третьего уровня — еще дальше от процессора.

Иерархическая структура памяти



Способы адресации

Размер адреса определяет максимальную область памяти, доступную для процессора.

Например, 16-разрядный адрес обеспечивает возможность использования 64Кбайт.

Хотя наиболее простой способ указания адреса - включение его в команду, это не эффективно.

Назначение способов адресации

1. Разрешить команде осуществлять доступ к ячейке памяти, адрес которой вычисляется во время выполнения(например массивы).
2. Манипулирование адресами в форме, подходящей для таких структур как стеки, массивы.
3. Запись полного адреса наименьшим количеством разрядов.
4. Вычисление адресов относительно местоположения команды.

Способы адресации

1. Неявная (подразумеваемая) адресация.
2. Непосредственная адресация.
3. Прямая адресация.
4. Косвенная адресация:
 - а) регистровая;
 - б) базовая;
 - в) индексная;
 - г) базовая индексная;
5. Относительная адресация.
6. Адресация устройств ввода/вывода.

Неявная адресация

Под неявной или подразумеваемой адресацией понимается такой метод адресации, когда в команде адрес операнда прямо не указывается, а подразумевается.

Сам адрес операнда процессором определяется однозначно по коду операции самой команды.

Например: STD – установить в 1 флаг DF.

Непосредственная адресация

При этом виде адресации операнд находится непосредственно в самой команде.

Например: MOV AX, 0FA2Bh – занести в регистр AX константу 0FA2Bh.

Прямая адресация

При этом способе адресации адрес операнда прямо указывается в команде.

Это указание определяется либо приведением имени регистра, где находится требуемый операнд (тогда, часто, такая адресация называется – прямой регистровой), либо указанием прямого адреса в ОЗУ. При программировании на языке ассемблера этот прямой адрес может выделяться квадратными скобками.

Например: `ADD AX, BX` – сложить содержимое регистров AX и BX; результат поместить в регистр AX или `MOV DL, MEM`

Косвенная адресация

Косвенная адресация является самым распространенным способом адресации данных, расположенных в ОЗУ. При косвенной адресации в команде указывается регистр, где находится адрес операнда, либо некоторое выражение по которому вычисляется этот адрес в ОП. При формировании адреса участвуют, в общем случае, базовые и индексные регистры, а также некоторые константы. Этот адрес обычно носит название *эффективного адреса* (**EA**). Различают несколько разновидностей косвенной адресации.

Регистровая косвенная адресация

- адрес размещен в указываемом в команде регистре.

Например: `ADD AX, [BX]` – сложить содержимое регистра AX и ячейки памяти ОЗУ, расположенной по адресу, указанному в регистре BX; результат поместить в регистр AX.

Базовая косвенная адресация

При базовой адресации эффективный адрес операнда определяется как сумма содержимого одного из базовых регистров плюс некоторая константа, называемая смещением в команде.

Например: `MOV AX, [BX+123h]` – передать в регистр AX содержимое ячейки памяти (слово) по эффективному адресу, определяемому как сумма содержимого базового регистра BX плюс константа 1234h.

Индексная косвенная адресация

может быть со смещением в команде, или без смещения.

Этот способ адресации очень удобен для адресации последовательности чисел в массиве.

Например: SUB AX, [SI+123h] – из содержимого регистра AX вычесть содержимое ячейки памяти ОЗУ по адресу, равному содержимому регистра SI плюс константа 123h; результат поместить в регистр AX.

Базовая индексная косвенная адресация

При этом способе адресации эффективный адрес (адресное смещение в сегменте) определяется как сумма содержимого базового и индексного регистров со смещением (константой) или без него. Этот способ адресации удобен для адресации элементов двумерных матриц.

Например: `MOV AX, [BX+SI+123h]` – переслать в регистр AX слово из сегмента DS O3Y по адресному смещению, вычисленному как сумма содержимого регистров BX, SI и константы 123h.

Относительная адресация

Этот способ адресации относится только к адресации команд. На линейных участках программы, адрес команды берется из регистра IP, т.е. имеет место косвенная регистровая адресация. При передачах управления адрес точки перехода определяется как сумма содержимого регистра IP и некоторого смещения. При этом команда становится короче и выполняется быстрее.

Например: JS 12h – если был перенос из старшего разряда, то – передать управление по указанному адресу.

Адресация ввода/вывода

используется для адресации устройств ввода/вывода только в тех компьютерах, в которых предусмотрено отдельное адресное пространство ввода/вывода. Так, он характерен для всех компьютеров, в которых используются процессоры Intel и совместимые с ними. В компьютерах Macintosh, использующих процессоры фирмы Motorola, таких команд нет. Там адресное пространство совмещено с общим адресным пространством ОЗУ и обращение к устройствам ввода/вывода такое же как к обычным ячейкам ОЗУ по командам MOV.